

Lane-Following Robotic Car (LFRC)

Final Report

Tyler Proctor

Department of Electrical Engineering and
Computer Science
Massachusetts Institute of Technology
Boston, MA, USA
proctort@mit.edu

Yaphet Lemiesa

Department of Electrical Engineering and Computer Science
Massachusetts Institute of Technology
Cambridge, MA USA
yaphk175@mit.edu

Abstract—We present a design for a Lane-Following Robotic Vehicle implemented entirely on an FPGA system, which streams the real-time vehicle’s camera output and the highlighted lane markings to an HDMI display. This robotic vehicle relies on the input of a camera sensor to locate upcoming line segments and perform calculations to remain within the lane boundaries, along with streaming this to an HDMI display utilizing a wireless transmitter. We implement this design using a standard robotic chassis and RealDigital Urbana FPGA Board, evaluate its performance using real test tracks and simulations for latency, and discuss potential areas for future expansion and improvement.

I. INTRODUCTION

Real-time lane detection is a critical component of autonomous vehicle navigation. While software-based approaches are common, they often suffer from high latency. FPGA-based solutions offer deterministic throughput but present their own challenges with resource management and timing closure, particularly when processing high-resolution video streams.

This project implements a lane detection pipeline on an AMD Spartan-7 FPGA. The primary technical challenges involved optimizing the Connected Component Labeling (CCL) for throughput, managing noise in the binary image, and resolving timing violations in the complex lane pairing logic. Additionally, fine-tuning the motor control for correct pulse width modulation (PWM) signals and correct ranges for movement proved to be a difficult task.

Our specific goals were to:

- 1) Achieve real-time processing at > 30 fps.
- 2) Robustly handle up to 4 distinct objects (connected components) to distinguish lane markings from noise.
- 3) Generate a stable midpoint calculation for steering.
- 4) Visualize the decision-making process via an HDMI overlay.

II. HARDWARE CONSTRUCTION

The hardware for this project comprises of:

- RealDigital Urbana Board with AMD’s Spartan-7 FPGA.
- Three-wheeled robotic vehicle chassis with secondary platform added on stilts to place the FPGA board.
- OJ3901 Motor Controller Board that handles communication between the FPGA board and the wheel motors.

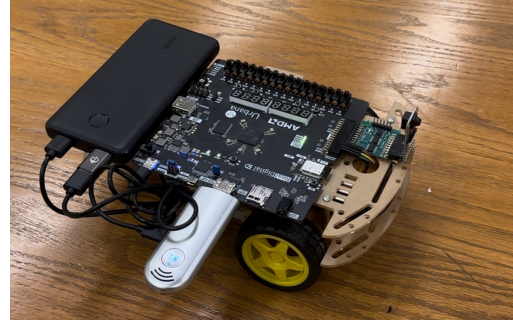


Fig. 1. The built robotic car and FPGA, which was the primary hardware for this project.

- Two DC Geared Motors to produce front 2 wheel drive.
- OV5640 Camera Sensor for navigation and capture of the vehicle’s point of view.
- TP30 HDMI Transmitter & Receiver which produce a wireless connection for displaying the lane detection system from the mobile robot to a standard HDMI screen.
- ANKER Portable Charger and Battery for mobile FPGA power supply.
- Pack of four AA batteries for motor and motor controller power.

III. IMAGE PROCESSING

Our project relies on the input image produced by the OV5640 camera sensor to interpret and interact with the surrounding environment. This image must be processed to extract the relevant lane features for the control system while simultaneously streaming a visual feed to the user. The processing pipeline is split into two primary clock domains: a high-speed 200 MHz domain, (`clk_camera`), for capturing raw sensor data and a 74.25 MHz domain, (`clk_pixel`), for processing and HDMI display. The pipeline consists of the following stages:

A. Pixel Reconstruction & Clock Domain Crossing

The `pixel_reconstruct` module deserializes the 8-bit parallel camera data, which is necessary for transforming the camera data into usable pixel data. To mitigate metastability

$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$
$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{8}$
$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{16}$

Fig. 2. Visualization of 3 x 3 Gaussian filter kernel, as used in our filtering module.

and bridge the asynchronous boundary between the camera’s varying PCLK and our system’s stable clock, we implement a Clock Domain Crossing (CDC) using a 2-stage asynchronous FIFO (`xpm_fifo_async`). This ensures that the downstream pipeline operates deterministically on the 74.25 MHz pixel clock.

B. Gaussian Blur Filter & DDR Buffering

Incoming pixels are processed by a 3×3 Gaussian Blur filter to reduce high-frequency noise, which is critical for anti-aliasing the lane edges. This is done by passing pixel data through four line buffers to utilize previous rows of pixel values in pair with a convolution module that performs the kernel convolution for each pixel. The filtered output is written to the DDR3 SDRAM via the `high_definition_frame_buffer` module. This off-chip memory serves two purposes: it acts as a frame buffer for the HDMI display (ensuring the user sees the full-resolution image) and allows the processing logic to read back pixel data via AXI streams.

C. YCrCb Conversion & Mask Threshold

To isolate the lane markings, we convert the RGB stream retrieved from memory into the YCrCb color space using a 3-cycle latency pipeline. We specifically target the Cr (Chrominance-Red) channel, as the pink lane tape exhibits high contrast in the red-difference spectrum compared to the gray road surface.

A hardware thresholder applies a binary mask to the Cr channel. Based on empirical testing, pixels are asserted as valid (logic high) if they fall within the range:

$$144 \text{ (0x90)} \leq Cr \leq 224 \text{ (0xE0)} \quad (1)$$

This specific bandpass filter effectively separates lane markings from the background floor and ambient lighting.

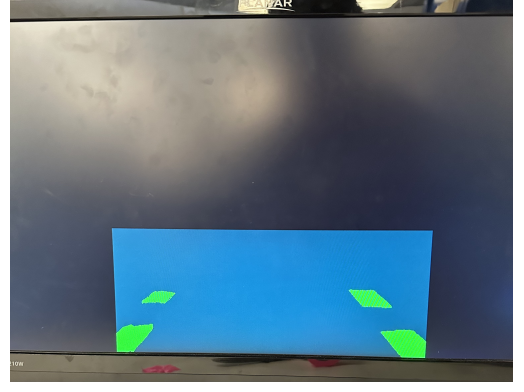


Fig. 3. Showcase of Region of Interest extracting the relevant section of pixels from the larger frame, focusing on only the upcoming lane markings. This display was produced with Mode 2 of the video output, as seen in section VII.

D. Downsampling & Region of Interest

To maximize throughput and minimize Block RAM (BRAM) usage in the subsequent warping stage, we down-sample the 1280x720 input by a factor of 2 in both directions, utilizing a bit-shift $\gg 1$.

Within this downsampled space, we define a static Region of Interest (ROI) to exclude the horizon and irrelevant sections of the camera input for lane recognition. The system only processes pixels within the source coordinates $x \in [296, 1004)$ and $y \in [432, 720)$. This results in an active processing window of 708×288 pixels. A dual-port BRAM buffer stores the binary mask of this region, allowing the warping engine to perform random-access reads of the mask bits.

E. Perspective Warping (IPM)

To facilitate accurate steering calculations, the camera’s perspective view is transformed into a top-down “Bird’s Eye View” (BEV). We utilize a software-defined Look-Up Table (LUT) architecture stored in Block RAM. The module iterates through target BEV coordinates (u, v) , retrieves the corresponding source coordinates (x, y) from the LUT, and fetches the pixel validity from the ROI buffer.

1) *Asynchronous Buffering Strategy*: The warping process can be bursty due to memory access patterns. To prevent pipeline stalls, the output of the warping module is buffered in a synchronous FIFO configured in First-Word Fall-Through (FWFT) mode. This configuration eliminates the read-latency cycle, ensuring that the Connected Component Labeling engine receives valid data immediately upon asserting its ready signal (`bev_fifo_m_ready`).

IV. LANE DETECTION

In order to perform steering and guidance for the robotic vehicle to remain within the lanes, a series of calculations must be made with the intention of detecting these lanes and finding a path between them, which is the job of our lane detection subsystem and is broken into the following parts:

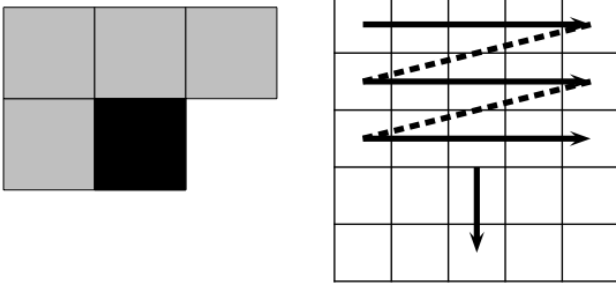


Fig. 4. The process of labeling connected components (CC) with 8-connectivity through a forward pass.

A. Connected Component (CC) Labeling

The Connected Component Labeling (CCL) module identifies and uniquely labels contiguous regions (connected components) of foreground pixels (where the binary value is 1) in the Bird's Eye View image. This is implemented using the **Two-Pass Single Storage (TPSS) architecture**.

1) *TPSS Architecture and Memory*: The TPSS architecture uses two passes over the image and a single BRAM for both image storage and equivalence table management.

- **Label ID**: A **12-bit Label ID** is used, allowing up to 4095 unique objects. This provides a necessary safety margin against label overflow caused by noise or texture in the road image.
 - **Port A** is used by the **Scanner** (Pass 1) for reading neighbor labels and writing the new label ID or merging conflicting labels.
 - **Port B** is used by the **Flattening** state machine (Pass 2) for reading the final label assignments.

2) *8-Connectivity Line Buffer*: To accomplish **8-connectivity** (checking pixels diagonally adjacent to the current pixel), the system must access labels for the Left, North, North-West, and North-East neighbors simultaneously. And since BRAM ports are limited, a dedicated **Line Buffer** is implemented in **Distributed RAM (LUTRAM)**. This buffer caches the resolved 12-bit labels of the row immediately above the current scan line ($y - 1$), enabling the necessary **single-cycle access** to all neighbor labels for efficient operation of the Scanner state machine.

3) *Retry Mechanism*: Due to the complexity of 8-connectivity, a single pixel can bridge up to four different labels. To handle this, the CCL system utilizes a **Retry State Machine**. When a conflict is detected, the pipeline stalls, the merge is performed in BRAM, and the pixel is re-evaluated until no conflicts remain.

Lastly, we testbenched this module with several patterns using binary 2d arrays. Referencing the paper, [3] we were able to test the specific designs and identify and uniquely assign labels to connected components.

B. Centroid Calculation & Normalization

The `cc_calculations` module accumulates the spatial x, y coordinates and total pixel area for each labeled component. To ensure timing closure before the complex pairing logic, the calculated centroids are registered into a buffer (`lane_h_avg_q`).

These centroids are computed in the downsampled BEV space. Before being used for display or final steering, they are upsampled by the downsampling factor (left-shifted by `DS_H_SHIFT`) and offset by the ROI origin ($x+296, y+432$) to map them back to the global 1280x720 coordinate system.

C. Lane Pairing & Stabilization Logic

The `lane_pair_filter` determines the steering trajectory by identifying the two largest blobs representing the left and right lane boundaries. A critical feature of our design is the **Midpoint Hold** mechanism.

$$x_{target} = \begin{cases} x_{new} & \text{if } valid_{curr} = 1 \\ x_{prev} & \text{if } valid_{curr} = 0 \end{cases} \quad (2)$$

If the system fails to detect a valid lane pair in a specific frame (e.g., due to glare or occlusion), the logic retains the valid midpoint from the previous frame (`valid_midpoints_hold`). This prevents steering jitter and ensures smooth control signals to the PWM drivers.

The filter sorts the connected components into left and right candidates based on the screen center. It selects the top-2 CCs per side based on **Area**. This ensures that larger, more significant lane markings are prioritized over small noise artifacts.

A weighted average is calculated between the pairs. To ensure more precise steering, bottom-heavy features are prioritized (3:1 weighting) over top features.

D. Guidance Calibrator

With the horizontal midpoint value produced before and knowledge of the general midpoint of the frame, the `motor_guidance` module calculates the delta between these two values and, using predetermined ranges from the frame midpoint (25 pixels on each side), decides if the robot should stay straight, slightly turn right, or slightly turn left. It then sends this command to the vehicle controls.

V. VEHICLE CONTROLS

Under direction of the decisions produced by the `motor_guidance` system, this subsystem handles communicating with the Motor Controller board to send commands for movement, utilizing a customized Pulse Width Modulation (PWM) module to send calculated signals to the two different motors. In order to account for the standard pixel clock cycles being too fast for PWM signals to be read by the motors, a larger period (255,000 instead of 255) and a bigger duty cycle have been implemented. This all is seen through the vehicle moving at a steady speed straight, or powering the wheels at different amounts to turn left or right.

VI. VIDEO OUTPUT & DEBUGGING

Visual feedback is essential for verifying the robot's decision-making process. The video pipeline mixes the raw camera feed with real-time utility overlays output via HDMI.

VII. VIDEO OUTPUT

To give the user better insight into the actual decision-making and the data that the robot is accumulating, a final video output is constructed to show on a normal HDMI screen. Due to the robot being mobile, an HDMI wireless transmitter and receiver are used to send this video output real time to a monitor.

A. Hardware-Selectable Debug Modes

To facilitate rapid testing and debugging without recompiling the bitstream, we implemented a video multiplexer controlled by the FPGA switches (`sw[5:1]`). This allows the user to inspect intermediate stages of the pipeline in real-time:

- **Mode 0 (Camera):** Raw RGB pass-through.
- **Mode 1 (Threshold):** Visualizes the binary mask (Black/White) to verify lighting sensitivity and Cr thresholding.
- **Mode 2 (ROI):** Highlights the active Region of Interest in green to verify the cropping window.
- **Mode 3 (Centroid Highlights):** Lane annotation overlay

B. Lane Annotation Overlay

The `lane_annotation` module performs the final rendering. It takes the calculated centroids, scales them back to screen coordinates, and overlays color-coded markers: Red for the left lane, Cyan for the right lane, and Yellow for the calculated target path. This overlay is alpha-blended with the source video to maintain visibility of the road conditions.

C. LED Telemetry

For immediate diagnostics without a screen, the design maps pipeline status to the on-board LEDs (`led[15:0]`). This includes a "Heartbeat" on LED[0], a "Lanes Valid" flag on LED[6], and a binary count of currently detected blobs on LEDs[14:12]. This allows for instant assessment of the CCL engine's load and tracking success.

D. HDMI Transmitter

Utilizing the HDMI Transmitter hardware, this muxed video signal is accessible to any normal HDMI screen that is wirelessly connected, allowing the robotic vehicle to move freely while still producing meaningful and real-time data and visuals to the screen.

VIII. EVALUATION (VIVADO & SYSTEM)

A. Timing

The design meets all timing constraints on the RealDigital Urbana target (Spartan-7).

- **Pixel Domain (74.25 MHz):** Post-route Worst Negative Slack (WNS) = 0.195 ns, Total Negative Slack (TNS) = 0.000 ns.
- **Camera Domain (200 MHz):** WNS = 0.363 ns.

The clocking architecture successfully closes timing across four distinct domains: Pixel (74.25 MHz), TMDS (371.25 MHz), Camera (200 MHz), and DDR (333 MHz).

B. Throughput & Latency

The active Bird's Eye View (BEV) region is 708×288 pixels (203 kpx). At a pixel rate of 74.25 Mpix/s, the CCL and Centroid pipeline processes a single frame in approximately 2.7 ms (> 350 fps theoretical throughput). The overall video path latency is dominated by the frame buffering required for the HDMI display. The critical path components include the Gaussian blur line buffer, DDR3 read/write operations, and the Warp fetch. The total system latency is approximately one frame plus line overhead which remains well within the < 33 ms real-time budget.

C. Resources (Post-Synthesis)

Utilization is bounded primarily by Block RAM (BRAM), which is used for the BEV Look-Up Table, Label Equivalence Table, and Frame Buffers.

- **LUTs:** 8,485 ($\approx 26\%$)
- **FFs:** 10,696 ($\approx 16\%$)
- **BRAM:** 68/75 (91%) – Primary Constraint
- **DSP:** 2/120 (1.7%)

To relieve BRAM pressure in future iterations, we suggest shrinking the ROI dimensions or implementing time-multiplexing for the memory access.

D. Power Consumption

Post-route vector-less estimation indicates a total on-chip power of **0.980 W** (0.901 W dynamic, 0.079 W static). The primary consumers are:

- **I/O (HDMI/Camera):** 0.535 W
- **DDR3 Controller:** 0.371 W
- **MMCM/Clocking:** 0.214 W

Current confidence in this estimate is Low due to the lack of switching activity files (SAIF/VCD).

E. Overheads vs. Benefit

- **DDR Frame Buffer:** Adds ≈ 1 frame of latency but guarantees timing-closed decoupling between the asynchronous camera and display clocks.
- **FWFT FIFO:** Implementing First-Word Fall-Through FIFOs after the warp stage removes the single-cycle "bubble" on CCL startup, improving pipeline utilization.
- **BEV LUT:** While expensive in BRAM usage, the Look-Up Table approach ensures deterministic warping throughput without requiring complex DSP arithmetic.

IX.

X. USE CASES AND COVERAGE

- **Supported:** Straight paths and gentle curves within the defined ROI; robust to sparse glare and noise via the CCL area filter and midpoint hold logic.
- **Occlusion Handling:** The system successfully retains the prior valid midpoint when `valid_midpoints` drops for a single frame.
- **Out of Scope:** Extreme curvature extending outside the ROI, multi-frame lane loss, and dynamic ROI adjustment.

XI. IMPLEMENTATION INSIGHTS

We replaced an early software-side warp with a BRAM-backed LUT to guarantee per-pixel determinism at the cost of BRAM (now 91% used), trading memory for timing closure and freeing DSPs for other tasks. BRAM usage proved to be one of our hardest challenges, as a variety of different modules and subsystems would depend on it for correct calculations with memory utilization. This meant we had to make trade-offs as to which modules could use more BRAM and which ones had to cut costs. This also led us to implemented the downsizing seen in the lane detection subsystem. We faced additional challenges with the video display, and correcting timing through pipelining in order to produce a stable display. This was especially difficult as a large number of our modules faced variable latency and used a nondeterministic number of clock cycles. This also led us to consolidate how our switch-driven HDMI mux worked, focusing on the important or relevant features of our system. Testing on a real track also proved to be harder than expected, as a number of factors unrelated to code precision always played a role.

XII. CHECKLIST

We were able to accomplish all of the commitments we set out for in our original checklist, from image processing to a working CCL module, to a final real-time output display. We were also able to accomplish our goals, showcasing a robotic car accomplish real lane detection and navigation. However, we were unable to accomplish the stretch goals that we listed or other additional features for this project.

AUTHOR CONTRIBUTIONS

Tyler Proctor: Top-Level integration, Image Processing pipeline, Centroid Calculation, and Lane Pairing logic, Guidance calculations, PWM testing, motor control integration, physical robot testing, and video demonstration. **Yaphet Lemiesa:** Developed the CCL 8-connectivity engine, Developed the Perspective Warping module, and HDMI Overlay logic. **Joint Work:** System debugging, timing optimization/pipelining, and writing final reports.

ACKNOWLEDGMENTS

We thank the entire course staff of 6.205 for their assistance and support in office hours. And a special thanks to Prof. Joe Steinmeyer for his constructive support and guidance during the project.

REFERENCES

- [1] M. Bilal, "Resource-efficient FPGA implementation of perspective transformation for bird's eye view generation using high-level synthesis framework," *IET Circuits, Devices & Systems*, vol. 13, no. 6, pp. 756–762, Nov. 2019, doi: 10.1049/iet-cds.2018.5263.
- [2] SunEdition, "Lane detection and turn prediction algorithm for autonomous vehicles," *Medium*, Dec. 2019. [Online]. Available: <https://medium.com/@SunEdition/lane-detection-and-turn-prediction-algorithm-for-autonomous-vehicles-6423f77dc841>
- [3] K. Schwenk and F. Huber, "Connected Component Labeling Algorithm for very complex and high resolution images on an FPGA platform," in *Proc. SPIE 9646, High-Performance Computing in Geosciences*, Oct. 2015, Art. no. 96460J.
- [4] A. Taylor, "FPGA Based Motor Control," *Hackster.io*, Jan. 16, 2023. [Online]. Available: <https://www.hackster.io/adam-taylor/fpga-based-motor-control-cd47e7>
- [5] Pololu Corporation, "DRV8835 Dual Motor Driver Carrier," Las Vegas, NV, USA. [Online]. Available: <https://www.pololu.com/product/2135/resources>
- [6] E. Taur, "Component-Labeler: A hardware implementation of component labeling," GitHub repository. [Online]. Available: <https://github.com/erictaur/Component-Labeler>